Toolkit T3: Agentic programming, exercise handout

Workshop and self-study exercises for the agentic-programming toolkit

Simon Scheidegger

HEC, University of Lausanne

Prompts: `lectures/lecture_06_B17_agentic_programming/code/exercise_prompts.md`
Solution notes: `lectures/lecture_06_B17_agentic_programming/code/exercise_solutions.md`

Materials:

https://github.com/sischei/Deep_Learning_for_Solving_And_Estimating_Dynamic_Economic_Models

Exercise Overview

Classroom track (Ex. 0–7, 110 min total incl. pre-flight):

#	Exercise	Time	Focus
0	Hello Claude (pre-flight)	10 min	First contact, zero setup
1	Environment Bootstrap	15 min	Setup & first Claude session
2	First Real Task	15 min	Level 3 prompts + Plan Mode
3	Prompt Refactor	15 min	6-component prompt framework
4	Write Your Own CLAUDE.md	10 min	Persistent project memory
5	Skills & Subagents	10 min	`/agents` UI; verifier subagent
6	Data → Figures → LaTeX	25 min	Mini research pipeline
7	Replicate a Method	10 min	Autonomous paper replication

Self-study extensions (Ex. 8–12):

#	Exercise	Time	Focus
8	Mincer Warm-Up Demo	15–30 min	End-to-end OLS pipeline
9	Specialist Subagent	20–45 min	Econ / MC / backtest reviewer
10	Hook & Real Skill	20–60 min	Hooks + `/strategic-revision`
11	Monte Carlo Study	30–45 min	Subagent-driven MC
12	**Ralph Autonomous Loop**	**30–45 min**	**Closed-loop autonomy**

Solutions in `lectures/lecture_06_B17_agentic_programming/code/exercise_solutions.md`; **consult only *after* attempting**.

Learning objectives

By the end of this lecture you should be able to:

- Compose effective prompts using role, context, task, constraints, output, and verification components
- Create reusable skills and subagents to decompose complex analysis workflows
- Build end-to-end pipelines from synthetic data generation through estimation, visualization, and LaTeX tables
- Use Bayesian active learning and Monte Carlo verification to test agentic reasoning robustness
- Replicate published computational results by extracting specifications and delegating multi-step tasks

Exercise 1

Environment Bootstrap

15 minutes

Exercise 1: Environment Bootstrap

Goal

Get a working Claude Code session in a version-controlled project.

1. Create `~/research/lecture-demo`
2. Copy in `generate_synthetic_data.py` and run it once
3. Initialize git and make an initial commit
4. Launch Claude Code and run `/init`
5. Ask Claude to create `explore_data.py` with `pathlib`
6. Run it on `data/synthetic_panel.csv`

Success: local data + working script + `CLAUDE.md` + git commit

Exercise 1: Solution Sketch

Reference answer

- Expected artifacts: `CLAUDE.md`, `data/synthetic_panel.csv`, `explore_data.py`
- `explore_data.py` should accept a CSV path and report shape, columns, dtypes, head, summary stats, and missingness
- The file should use `pathlib.Path`, not `os.path`

Common failure

Students create the script before the data exists locally. Fix that by copying and running `generate_synthetic_data.py` first.

Exercise 2

First Real Task

15 minutes

Exercise 2: First Real Task

Goal

Use the Level 3 prompt pattern on the shared synthetic panel.

```
> I have a dataset at data/synthetic_panel.csv.
  Without loading it yet, first tell me how to inspect it.
  Then:
  1. Load it and print diagnostics
  2. Compute annual average revenue by treatment group
  3. Generate a 2x2 panel of plots
  4. Save the figure to figures/panel_overview.pdf
  5. Write a 3-sentence description to notes/data_description.md
```

Reflect: Where did Claude excel? Where did it need correction?

Exercise 2: Solution Sketch

Expected artifacts

- `figures/panel_overview.pdf`
- `notes/data_description.md`
- A short diagnostic printout in the terminal

Typical interpretation

- Revenue trends are similar pre-2015
- Post-2015 treated firms drift modestly upward
- Log revenue and log employees are positively related

Exercise 3

Prompt Refactor

15 minutes

Exercise 3: Prompt Refactor

Goal

Transform a bad prompt using the 6-component framework.

Bad prompt: `> Analyze the data and run the regression and make a table.`

Task:

1. Diagnose the missing components
2. Rewrite as Level 3
3. Compare the vague and rewritten versions in Claude Code

Exercise 3: Solution Sketch

```
> [ROLE] You are a computational economist working on panel data analysis.
  [CONTEXT] I have a firm-level panel dataset at data/synthetic_panel.csv.
  Variables: revenue, employees, treated, firm_id, year. Treatment starts in 2015.
  [TASK] Estimate log(revenue) ~ treated * post + firm_FE + year_FE
  with standard errors clustered at the firm level.
  [CONSTRAINTS] Use pyfixest. Do not modify the data file.
  [OUTPUT] Save regression results to results/did_estimates.json
  and a LaTeX table to paper/tables/tab_did.tex.
  [VERIFY] Also test on synthetic data where true effect = 0.05.
```

Key

Every component is explicit. Two different students should be able to read this prompt and ask Claude for essentially the same deliverable.

Exercise 4

Write Your Own CLAUDE.md

10 minutes

Exercise 4: Write Your Own CLAUDE.md

Goal

Create a project memory file and test both normal behavior and guardrails.

1. Copy `lectures/lecture_06_B17_agentic_programming/code/CLAUDE_md_template.md`
2. Fill in Repository Structure, Coding Standards, and Current Status
3. Start a fresh Claude session and ask for a project summary
4. Test one normal coding request
5. Test one forbidden request touching a read-only path

Exercise 4: Solution Sketch

What success looks like

- Claude uses your notation, folder structure, and coding rules immediately
- Claude respects `pathlib`, type hints, and naming conventions
- Claude refuses or pushes back on edits to `data/raw/`

High-value sections

`Repository Structure`, `Coding Standards`, and `Current Status` have the highest immediate payoff.

Exercise 5

Skills & Subagents

10 minutes

Exercise 5: Create a Skill & Use Subagents

Goal

Create one reusable skill and one verifier-style subagent.

Part A: Skill

1. Copy `lectures/lecture_06_B17_agentic_programming/code/skills/example_06_B17_agentic_programmin`
2. Save it to `.claude/skills/data-diagnostics/SKILL.md`
3. Run `/data-diagnostics data/synthetic_panel.csv`

Part B: Subagent

1. Copy `lectures/lecture_06_B17_agentic_programmin` `code/subagents/example_06_B17_agentic_programmin` `verifier.md`
2. Save it to `.claude/agents/verifier.md`
3. Ask Claude to use the verifier subagent on the diagnostics report

Exercise 5: Solution Sketch

Expected artifacts

- `.claude/skills/data-diagnostics/SKILL.md`
- `.claude/agents/verifier.md`
- Diagnostics note in `notes/`

Conceptual distinction

- **Skill**: repeatable multi-step workflow
- **Subagent**: isolated verifier or researcher
- Use subagents when you want context separation

Exercise 6

Data $\rightarrow$ Figures $\rightarrow$ LaTeX

25 minutes

Exercise 6: Build a Mini Pipeline

Goal

Build a pipeline from shared synthetic data to a publication-ready figure and LaTeX table.

1. Run `/data-diagnostics data/synthetic_panel.csv`
2. Write `code/estimate_did.py`
3. Write `code/plot_did.py`
4. Write `code/make_table.py`
5. Verify the estimated effect against the true value 0.05

Stretch: add `run_all.py` with a `-dry-run` flag.

Exercise 6: Solution Sketch

Expected artifacts

- `results/did_results.json`
- `figures/event_study.pdf` and `.png`
- `paper/tables/tab_did.tex`
- Optional `run_all.py`

Minimum verification

- Two-way FE coefficient near 0.05
- Statistically significant at the 5% level
- Pre-trend coefficients near zero

Exercise 7

Replicate a Method

10 minutes

Exercise 7: Paper Replication

Goal

Use an AI agent to replicate a published result.

Choose: **Beginner** (Lasso, Stock&Watson 2012) | **Intermediate** (SINDy, Brunton 2016) | **Advanced** (DEQN from earlier lectures)

```
> Role: Replication engineer.
  Paper: [title]. Mission: Replicate [figure/table].
  Rules: no author code; track notes/todo.md; commit after each step.
  Before implementing: extract figure axes, ranges, and CIs.
  Write plan to notes/plan.md. Ask me to confirm.
```

Exercise 7: Solution Sketch

Success criteria

- A bounded target figure or table
- Progress tracked in `notes/plan.md` and `notes/todo.md`
- Final comparison written to `notes/replication_assessment.md`

Stop rule

If the agent is stuck after three serious attempts, write `notes/blocked.md` and stop. "The code runs" is not the same as "the result matches."

Appendix

Essential Prompt Library

Prompt Library

Session:

```
> Write notes/session_YYYY-MM-DD.md:
  what we did, worked, failed, next.
```

Code Review:

```
> Review .py files. Refactor >50-line
  functions. Add missing docstrings.
```

Tests:

```
> Write pytest tests with known-answer
  synthetic data.
```

Debug:

```
> Error: [paste]. Don't fix yet.
  Explain cause. Propose 2 fixes.
```

Data:

```
> Load [file]. Do NOT modify.
  Print shape, dtypes, missing%,
  summary stats. Flag outliers.
```

Merge:

```
> Left join [A] with [B] on [key].
  Assert no rows dropped.
```

Writing:

```
> Improve clarity. Don't change
  meaning or citations.
```

Citations:

```
> Do NOT invent citations.
  Only cite papers in references.bib.
```